

View DFC Overview

An DFC of the View type is an internally integrated part of any View (except for [Header View](#), where a View DFC is not required).

In most cases, one View DFC is linked to only one View and its name should be the same as the name of the View. However, the GenericPortalTab and GenericPortalForm View DFCs for Views of [Form](#) type and [Tab View Template](#) can be linked to many Views of the respective type.

View DFCs are not typically created from scratch, and using [View Templates](#) instead is recommended. When creating a View from a Template, its View DFC is also copied (duplicated).

View DFC Features

The View DFC is responsible for rendering a part of the Screen. There are a few main assumptions for this DFC:

- ▶ It should contain only one Step rendering the User Interface
- ▶ It should NOT contain any business logic or handling of the events
- ▶ It should be able to load data from the database for display purposes
- ▶ It should not be able to modify data (an DFC of the View type cannot perform database actions such as "update", "insert" or "delete". These operations are not invoked in a transaction, which can lead to unexpected behavior of the database when an error occurs).
- ▶ It must return an External Output Action

Typical View DFC Structure

A View DFC contains the elements described below.

Interface

The interface is configured in the [DFC Interface Tab](#).

Properties

General Work Instructions Advanced **Interface**

☒ Enable Interface

Inputs

Name	Data Type	Required
Screen	Char	☒
ScreenTitle	Char	☒
ViewTitle	Char	☒
DefaultAction	Char	☒

Test Value

Default Value

Outputs

Name	Data Type
Action	Char

These are the External Inputs available:

- ▶ **Screen** – the screen code of Screen for which this View is being displayed
- ▶ **ScreenTitle** – the title of the Screen (already formatted)
- ▶ **ViewTitle** – the title of the View (already formatted)
- ▶ Any Portal Session Variable

These are the External Outputs available:

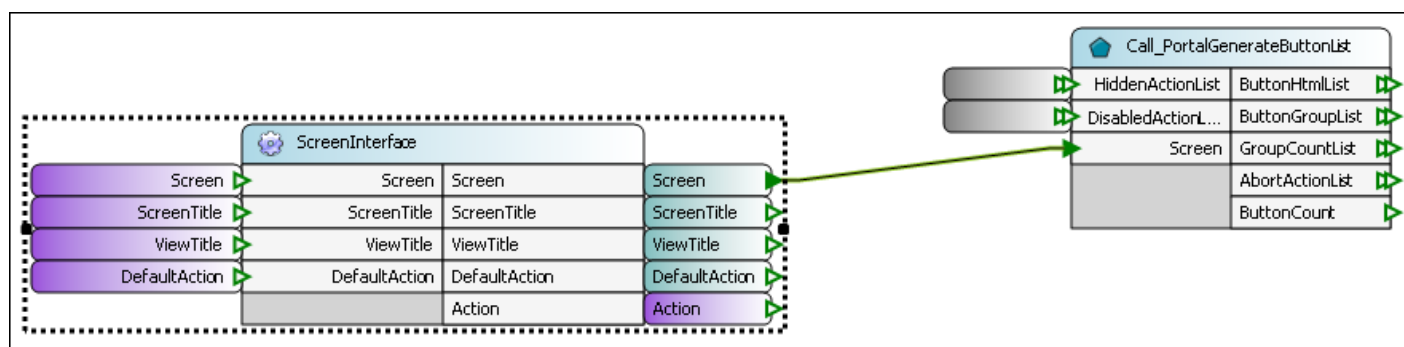
- ▶ Typical:
 - ▷ **Action** (required) – the Action triggered on this View
 - ▶ If there was no Action triggered from this View, an empty string is returned
- ▶ Additional:
 - ▷ **DefaultAction** – the default Action of this View
 - ▶ If the View should not react to such events as pressing the ENTER key or a page refresh, leave this value empty
 - ▷ **Any other External Output** – will be merged automatically to the Portal Session if this View DFC triggered a submit of the Screen
 - ▷ **MergeOutputs**
 - ▶ If the submit was not triggered by this View DFC on the Screen and merging additional External Outputs to the Portal Session is required, then MergeOutputs must be set to true
- ▶ Advanced:
 - ▷ **ToStackIndex** and **ToScreen** – enable controlling the Screen flow without any Actions (for more information, see [Screen Flow Processing Diagram](#))
 - ▶ Please note that the best practice is to always navigate using the Action configuration

View DFCs are not able to pass Outputs with the "**External_**" prefix. See [Portal Session Variables Handling](#) for more information.

Step Containing the User Interface

There must be one Step that displays the parts of the UI to the user:

- ▶ **ScreenInterface** Function – the result of using the HTML Layout Editor (which is recommended)
- ▶ It can contain the **PortalGenerateButtonList** DFC
- ▶ It can contain a Business Control (which does not exist in this example) (for more information, see [Using Standard Business Controls in a View](#) below)
- ▶ Additional Functions (e.g., to load data from the database for display purposes)



The **PortalGenerateButtonList** DFC generates a list of HTML strings to be injected directly into the HTML Layout Editor. Each element of this list contains all the Actions configured as Buttons for the current View of a given Action Group. The buttons are limited to the ones that are allowed for the current Screen and the [Role](#) of the current employee. Additionally, you can manually hide or disable some of the Buttons by passing their names to the HiddenActionList and DisabledActionList Inputs.

The actual display of the Buttons happens in the **ScreenInterface** Function of the display Step:

```
<!-- view title -->
<h1 class="Label">{viewTitle}</h1>
<hr/>
<!-- Put your screen controls here -->

<!-- Portal Buttons -->
<div class="ButtonList" data-flx-iterate="{i.ButtonHtmlList}" data-flx-
filter="{ButtonGroupList[i]} == ''">{ButtonHtmlList[i]:html}</div>
```

By default, the View Template uses all the Buttons without a group and displays them at the bottom of the View UI. You can additionally adjust this by moving the related HTML code to a different place. If you want to display two sets of Buttons (different groups) in different places, copy the buttons' HTML code to the desired place and modify the data-flx-filter expression to match your group (the **Group Tag** property on the [View Action: General Tab](#)). For example: data-flx-filter="{ButtonGroupList[i]} == 'TOP'".

You cannot display an individual Button from the same group in a different position. This is why you should create as many distinct groups as there are places where you wish to display the Buttons.

The sample usage of this Function can be found in the PortalPaging_View Template. The PREV and NEXT buttons are hidden if the first or last record is displayed.

Using Standard Business Controls in a View

You have to define a single View that will only contain the required Business Control. The Business Control's height and width must both be set to 100%. The View must be embedded in a unique and dedicated Panel. This will help to support the auto-sizing of the Screen when using different devices with different resolutions.

Not all standard Business Controls support size in percentages. For details, go to each Business Control's description in the [Business Controls](#) section of this Help.